

Array

1.Problem Statement(Easy)

We are given two variables x and n . Implement $\text{pow}(x, n)$, which calculates x raised to the power n (i.e., x^n).

Example 1:

Input: $x = 2.00000$, $n = 10$

Output: 1024.00000

Example 2:

Input: $x = 2.10000$, $n = 3$

Output: 9.26100

Example 3:

Input: $x = 2.00000$, $n = -2$

Output: 0.25000

Approach:

We need to compute x^n , and instead of multiplying x by itself n times, we break it down to smaller parts using the following approach:

1. Even Exponent (n is even) :

We can express x^n as $(x^{\{n/2\}})^2$. This reduces the problem to finding $x^{\{n/2\}}$, which we then square to get x^n .

2. Odd Exponent (n is odd):

For odd n , we split it as $x^n = x * x^{\{n-1\}}$. Now, we can compute $x^{\{n-1\}}$ as $(x^{\{(n-1)/2\}})^2$ and multiply by x .

By repeatedly halving the exponent, we drastically reduce the number of multiplications, making this approach much more efficient than directly multiplying x by itself n times.

The base case is reached when $n == 0$ and returns 1.

Negative Exponent: If n is negative, compute the result for $-n$ and take the reciprocal.

Divide and Conquer: Recursively compute $x^{(n/2)}$. If n is even, square the result. If odd, multiply by x once more.

Return: Reciprocal the result for negative n ; otherwise, return the computed value.

2. Problem Statement (Medium)

Given an array `nums` with `n` objects colored red, white, or blue, sort them in place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0 , 1 , and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example: Toy Sorting

In a cheerful playroom, kids played all day with Red Cars, White Teddy Bears, and Blue Balls, leaving the toy box in a huge mess. The next morning, they struggled to find their favorite toys. Seeing this, the caretaker had a clever idea—instead of using extra bins, they neatly arranged the toys inside the same box. Red Cars first, Teddy Bears in the middle, and Blue Balls at the end—all sorted in no time! The kids were thrilled, and from that day on, keeping the toy box tidy became a fun and easy habit!

Example 1:

Input: `nums = [2,0,2,1,1,0]`

Output: `[0,0,1,1,2,2]`

Example 2:

Input: `nums = [2,0,1]`

Output: `[0, 1, 2]`

Approach:

Using Dutch National Flag Algorithm :

Using three pointers: low, mid, and high. We sort the array, where all 0s are at the front, 1s are in the middle, and 2s are at the end. This avoids extra space and ensures optimal efficiency.

1. Initialize low = 0, mid = 0, and high = n - 1.

2. Traverse the array while mid <= high:

If `nums[mid] == 0`, swap with `nums[low]` and increment both low and mid.

If `nums[mid] == 1`, just move mid forward.

If `nums[mid] == 2`, swap with `nums[high]` and decrement high (don't move mid yet).

3. Stop when mid > high. The array is now sorted in-place.

3. Problem Statement 2213(Hard)

Augmenting a Segment Tree

You are given a 0-indexed string s . You are also given a 0-indexed string $queryCharacters$ of length k and a 0-indexed array of integer indices $queryIndices$ of length k , both of which are used to describe k queries.

The i th query updates the character in s at index $queryIndices[i]$ to the character $queryCharacters[i]$.

Return an array $lengths$ of length k where $lengths[i]$ is the length of the longest substring of s consisting of only one repeating character after the i th query is performed.

Example 1:

Input: $s = \text{"babacc"}$, $queryCharacters = \text{"bcb"}$, $queryIndices = [1,3,3]$

Output: $[3,3,4]$

Explanation:

- 1st query updates $s = \text{"bbbacc"}$. The longest substring consisting of one repeating character is "bbb" with length 3.
 - 2nd query updates $s = \text{"bbbccc"}$.
The longest substring consisting of one repeating character can be "bbb" or "ccc" with length 3.
 - 3rd query updates $s = \text{"bbbbcc"}$. The longest substring consisting of one repeating character is "bbbb" with length 4.
- Thus, we return $[3,3,4]$.

Example 2:

Input: $s = \text{"abyzz"}$, $\text{queryCharacters} = \text{"aa"}$, $\text{queryIndices} = [2,1]$

Output: $[2,3]$

Explanation:

- 1st query updates $s = \text{"abazz"}$. The longest substring consisting of one repeating character is "zz" with length 2.
 - 2nd query updates $s = \text{"aaazz"}$. The longest substring consisting of one repeating character is "aaa" with length 3.
- Thus, we return $[2,3]$.

Constraints:

$1 \leq s.length \leq 105$

s consists of lowercase English letters.

$k == \text{queryCharacters.length} == \text{queryIndices.length}$

$1 \leq k \leq 105$

queryCharacters consists of lowercase English letters.

$0 \leq \text{queryIndices}[i] < s.length$

Hints:

1. Use a segment tree to perform fast point updates and range queries.
2. We need each segment tree node to store the length of the longest substring of that segment consisting of only 1 repeating character.
3. We will also have each segment tree node store the leftmost and rightmost character of the segment, the max length of a prefix substring consisting of only 1 repeating character, and the max length of a suffix substring consisting of only 1 repeating character.
4. Use this information to properly merge the two segment tree nodes together.

Approach:

After each character update, find the longest run of identical characters in the string. Brute force $O(n)$ per query is too slow for the given constraints.

Solution: Segment Tree

What Each Node Stores

Every node covering a segment stores:

maxLen — longest run of identical chars in this segment

prefLen — longest run starting from the left end

sufLen — longest run ending at the right end

leftChar / rightChar — characters at both ends of the segment

The prefix and suffix lengths are needed because when two segments are merged, a new run can form across the boundary between them.

Merging Two Nodes

When combining left and right child nodes:

- Boundary run forms only if $\text{left.rightChar} == \text{right.leftChar}$, giving $\text{left.sufLen} + \text{right.prefLen}$
- $\text{maxLen} = \max$ of left's maxLen, right's maxLen, and the boundary run
- prefLen extends into the right side only if the entire left segment is one character and it matches right's leftChar
- sufLen extends into the left side only if the entire right segment is one character and it matches left's rightChar

Build, Update, Query

- Build: Leaves are individual characters. Internal nodes are built bottom-up by merging children — $O(n)$
- Update: On each query, update the leaf at the given index, then remerge all ancestors on the path back up — $O(\log n)$
- Query: The root node's maxLen is always the answer after each update — no separate traversal needed

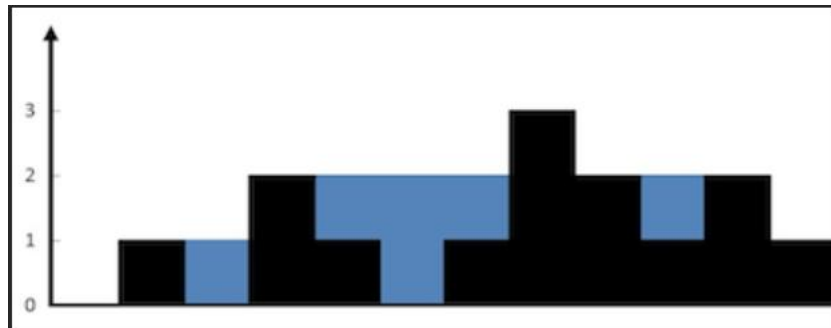
4. Trapping Rain Water Problem 42:

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Trapping Rain Water : Understanding the Problem

Imagine a row of buildings of different heights standing side by side. After it rains, water collects in the gaps between the buildings wherever shorter ones are surrounded by taller ones. The depth of these holes determines how much water is trapped. There won't be any water trapped if the buildings are exactly aligned and have no dips. But if there are valleys between taller buildings, water fills those spaces. We have to find out how much water can be stored.

Example 1:



Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:

Input: height = [4,2,0,3,2,5]

Output: 9

Constraints:

$n == \text{height.length}$

$1 \leq n \leq 2 * 10^4$

$0 \leq \text{height}[i] \leq 10^5$

Approach:

Using Two-Pointer Approach :

1. Initialize Variables:

lmax stores the maximum height encountered from the left.

rmax stores the maximum height encountered from the right.

low starts from 1 and moves right, while high starts from n-2 and moves left as at position 0 and n-1 we are unable to store water over there.

2. Iterate Using Two Pointers:

Continue iterating while low <= high to find out the total water we can store.

3. Update Maximums:

Update lmax as the maximum of lmax and height[low] to ensure that any trapped water depends on the tallest bar to the left.

Update rmax as the maximum of rmax and height[high] to ensure that water depends on the tallest bar to the right.

4. Calculate Water Trapped:

If lmax <= rmax, add lmax - height[low] to the total, since water trapped at low depends on lmax, and increment low. Otherwise, add rmax - height[high] to the total and decrement high.

5. Return Result:

Return the total trapped water stored in ans.

Problem 5: Sliding Window Problem

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window.

Example 1:

Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`

Output: `[3,3,5,5,6,7]`

Explanation:

Window position Max

`[1 3 -1]` -3 5 3 6 7 3

1 `[3 -1 -3]` 5 3 6 7 3

1 3 `[-1 -3 5]` 3 6 7 5

1 3 -1 `[-3 5 3]` 6 7 5

1 3 -1 -3 `[5 3 6]` 7 6

1 3 -1 -3 5 `[3 6 7]` 7

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Approach

Sliding Window using Monotonic Deque :

1. Set up the deque:

Create a double-ended queue (deque) to store indices of elements in the current window — not the values themselves. The idea is to keep the deque in a way that the front always holds the index of the maximum element in the window.

2. Build the first window:

For the first k elements, go through each one and remove indices from the back of the deque if their values are smaller than the current element.

Then, add the current index to the back of the deque.

After the first window is built, the front of the deque holds the index of the largest value in that window.

3. Start sliding the window:

From index k onward, for each new element:

First, remove the front index if it has fallen out of the current window range.

Then, again remove smaller values from the back of the deque, just like before.

Add the current index to the back of the deque.

Now the front holds the index of the maximum in the current window, so push its value into the result list.

4. Return the result:

After sliding through the entire array, return the list containing the maximum values for each window.

Problem 6: Kadane's Algorithm

Given an integer array `nums`, find the subarray with the largest sum and return its sum.

Example:

Ravi loves coding and wants to solve problems on LeetCode every day to improve his skills. Some days he solves many problems (+points), while on tough days, he struggles or skips (-points). To find his best streak, he uses Kadane's algorithm, which helps track the highest consecutive score. Instead of checking all days manually, the algorithm smartly continues a streak or starts fresh when needed.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`

Output: 6

Explanation: The subarray `[4,-1,2,1]` has the largest sum of 6.

Example 2:

Input: `nums = [1]`

Output: 1

Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5, 4, 1, 7, 8]`

Output: 25

Explanation: The subarray `[5, 4, 1, 7, 8]` has the largest sum of 25.

Approach:

Using Kadane's Algorithm :

1. Kadane's Algorithm has two choices at every element :

Add the current element to continue the current subarray.

Start the new subarray, If the current element gives a higher value than the previous subarray sum.

2. It tracks the sum of the current subarray (currSum) and the highest sum found so far (maxSum).

3. Initialize currSum and maxSum to nums[0].

4. Iterate through the array, updating currSum to the maximum of:

Starting a new subarray with nums[i], or

Adding nums[i] to the current subarray.

5. Update maxSum with the larger of maxSum and currSum.

6. Return maxSum as the result.

Dry Run

Final Answer:

Maximum Subarray Sum = 6